Universidad Nacional del Sur

Arquitectura y Diseño de Sistemas — 2026

Packeteer

Diagramas de Arquitectura

Documentación del Proyecto

Práctica de Arquitectura y Diseño de Sistemas 2026

Grupo N°18

Diego Galan · Galo Mercére · Ismael Lopez · Matias Junca · Nicolas Bernardini

1. Introducción y Descripción del Sistema

Packeteer es una plataforma de logística colaborativa diseñada para conectar a personas que necesitan enviar paquetes con conductores independientes dispuestos a transportarlos. Su objetivo central es optimizar y descentralizar el proceso de envíos, brindando una alternativa ágil y flexible frente a los servicios de mensajería tradicionales mediante el aprovechamiento de la capacidad ociosa de los vehículos en circulación.

Para los usuarios particulares, el sistema ofrece una experiencia completa donde pueden cotizar el costo de un envío de manera dinámica, gestionar el pago y realizar un seguimiento de la ubicación de su paquete en tiempo real.

Por otro lado, le aporta gran valor a los conductores al permitirles monetizar sus viajes. El sistema les otorga la libertad de explorar y aceptar entregas que se ajusten a su ubicación actual y a la capacidad de carga de su vehículo. Adicionalmente, la plataforma garantiza un entorno seguro y confiable a través de un sistema de calificaciones y un módulo de soporte dedicado.

1.1 Actores del sistema

Actor 1 - Remitente: Es el usuario particular que utiliza la plataforma con la necesidad de enviar paquetes. Sus acciones principales incluyen cotizar el costo de los envíos ingresando origen y destino, pagar el servicio, monitorear la ubicación de su paquete en tiempo real durante todo el trayecto y, opcionalmente, abrir tickets de soporte o calificar al conductor al finalizar la entrega.

Actor 2 - Transportista: Es el conductor independiente registrado en la plataforma con un vehículo activo. Sus acciones principales consisten en explorar la lista de viajes disponibles en su zona (filtrados por la capacidad de su vehículo), aceptar viajes, actualizar el estado de la entrega en sus distintas etapas y recibir su remuneración.

Actor 3 - Soporte técnico: Es el personal interno o administrativo de la plataforma encargado de velar por la calidad del servicio. Sus acciones principales son gestionar, responder y dar resolución técnica a los tickets de reclamos originados por los remitentes, además de visualizar métricas globales y analíticas del negocio.

1.2 Sistemas externos

Clerk: Servicio externo de gestión de identidades y autenticación. El sistema interactúa con Clerk para registrar usuarios de forma segura, gestionar sus inicios de sesión y manejar los perfiles y roles. Tanto el frontend como el backend del sistema consulta a Clerk para autenticar, y Clerk devuelve la validación y datos del perfil.

Mapbox (API de Mapas): Proveedor de servicios de geolocalización y cartografía. El sistema lo consume para validar que las direcciones ingresadas existan, renderizar el mapa para el seguimiento en tiempo real y calcular rutas. Flujo de datos: El sistema lo llama (envía coordenadas/direcciones) y recibe validaciones y mapas.

Mercado Pago (API de Pagos): Pasarela externa de procesamiento de pagos. Se utiliza para cobrar de forma segura a los remitentes cuando confirman la solicitud de un envío. *Flujo de datos:* Bidireccional (el sistema envía la solicitud de cobro con el monto, y Mercado Pago notifica al sistema el estado del pago mediante webhooks).

Open-Meteo (API Meteorológica): Proveedor de datos del clima en tiempo real. Al momento de cotizar un envío, el sistema lo utiliza para consultar el estado del tiempo en la ubicación de origen, registrando esta condición climática en la base de datos. *Flujo de datos:* El sistema llama al externo (envía latitud/longitud y recibe los datos climáticos actuales).

1.3 Requerimientos funcionales y no funcionales

Requerimientos funcionales:

RF00 - Registro y Autenticación: El sistema debe permitir el registro e inicio de sesión de usuarios, delegando la captura y validación de credenciales (contraseñas) a Clerk.

RF01 - Cotizar Envío: El sistema debe permitir al remitente ingresar origen, destino y tamaño del paquete para visualizar un precio estimado antes de confirmar.

RF02 - Confirmar y Publicar: Tras aceptar la cotización, el sistema debe procesar el pago y crear el registro del envío asignándole el estado inicial "Buscando".

RF03 - Seguimiento en Tiempo Real (Tracking): El sistema debe permitir al remitente visualizar en un mapa interactivo la ubicación GPS actualizada del transportista asignado, habilitando únicamente mientras el envío permanezca en estado "En Camino".

RF04 - Calificar Servicio: El sistema debe habilitar al remitente a dejar una puntuación (1 a 5) y una reseña escrita al transportista únicamente cuando el envío haya alcanzado el estado "Entregado".

RF05 - Abrir Ticket de Soporte: El sistema debe permitir al remitente generar un reclamo asociado a un envío específico, siempre y cuando este ya haya sido asignado a un transportista (estado "Aceptado" o superior).

RF06 - Historial de Envíos: El sistema debe permitir al remitente visualizar el listado de todos sus pedidos realizados, detallando estado, fecha y costo.

RF07 - Historial de Tickets: El sistema debe permitir al remitente visualizar todos los tickets generados, detallando estado actual, agente de soporte técnico asignado asignado y resolución (en caso de estar cerrado).

RF08 - Alternar Disponibilidad: El sistema debe proveer un control (interfaz) que le permita al transportista activar o desactivar su estado para comenzar a recibir notificaciones de viajes.

RF09 - Explorar Viajes: Cuando el transportista esté disponible, el sistema debe listarle los envíos en estado "Buscando" que se encuentren en su radio geográfico y cuyo tamaño no exceda la capacidad de su categoría de vehículo.

RF10 - Aceptar Viaje: El sistema debe permitir al transportista seleccionar un envío de la lista de disponibles y confirmar su aceptación, vinculando su usuario al viaje y cambiando automáticamente el estado a "Aceptado".

RF11 - Navegación / Hoja de Ruta: El sistema debe proveer al transportista un mapa interactivo que trace la ruta sugerida desde su ubicación actual hacia el punto de retiro (origen) y, posteriormente, hacia el punto de entrega (destino).

RF12 - Actualizar Estado del Viaje: El sistema debe permitir al transportista registrar el progreso físico del envío, marcándolo primero como "Retirado" y finalmente como "Entregado".

RF13 - Historial de Viajes: El sistema debe permitir al transportista consultar el registro de sus servicios completados y el total de ganancias acumuladas.

RF14 - Tomar Ticket de la Cola: El sistema debe listar los tickets en estado "Pendiente" y permitir que un agente de soporte técnico se asigne uno a sí mismo, cambiando el estado a "En Proceso".

RF15 - Resolución de Incidentes: El sistema debe permitir al personal de soporte tecnico registrar una conclusión técnica, agregar comentarios y cambiar el estado del reclamo a "Resuelto".

RF16 - Consultar Historial de Soporte: El sistema debe permitir al personal de soporte tecnico visualizar todos los tickets que él mismo resolvió anteriormente.

RF17 - Validar Concurrencia de Viajes: El sistema debe bloquear la acción de marcar un paquete como "Retirado" si el transportista ya posee otro envío en tránsito, asegurando la exclusividad del viaje.

RF18 - Calcular Tarifa Base: El sistema debe calcular en tiempo real el precio del envío cruzando la distancia de la ruta, las condiciones climáticas del momento y el multiplicador de la categoría del paquete.

RF19 - Calcular Liquidación: El sistema debe calcular el monto final a pagar al transportista, aplicando el porcentaje de ganancia base y sumando bonificaciones dinámicas por Zona Caliente y Puntuación Promedio.

RF20 - Definir Zonas Calientes: El sistema debe ejecutar un proceso analítico sobre el historial reciente de viajes para delimitar las áreas geográficas de alta demanda y establecer su franja horaria de vigencia.

RF21 - Generar Métricas Globales: El sistema debe consolidar periódicamente las estadísticas de la plataforma (cantidad total de envíos, ganancia neta retenida) para nutrir el panel de administración.

RF22 - Liquidación de Pagos: El sistema debe ejecutar un proceso batch periódico (ej. nocturno) para agrupar todos los saldos en estado "Liberado" de cada transportista y gestionar la transferencia consolidada hacia sus cuentas bancarias externas, cambiando el estado de dichos pagos a "Liquidado".

RF23 - Recalcular Puntuación de Transportista: Tras la recepción de una nueva calificación (RF04), el sistema debe recalcular de forma automática e incremental la puntuación promedio del transportista y actualizar su cantidad total de reseñas en la base de datos.

RF24 - Gestión de Vehículo y Perfil: El sistema debe permitir al Transportista registrar los datos técnicos de su vehículo (patente, categoría) por primera vez, así como visualizar y actualizar esta información posteriormente.

Requerimientos no funcionales:

RNF01: El sistema debe integrarse con APIs externas (Geolocalización/Clima/Financieras) mediante protocolos REST/JSON. REST y JSON son el estándar actual de la industria, lo que garantiza una integración ligera, rápida y compatible con prácticamente cualquier proveedor de servicios externos.

RNF02 (Integración): El procesamiento analítico debe ejecutarse en horarios de baja concurrencia para no afectar el tiempo de respuesta transaccional. Los cálculos masivos consumen muchos recursos de la base de datos. Hacerlo en diferido asegura que el sistema principal no se ralentice para los usuarios

RNF03 (Patrones de Diseño): El backend del sistema debe estar estructurado siguiendo el patrón de arquitectura MVC (Modelo-Vista-Controlador) y respetando los principios SOLID para garantizar su escalabilidad y mantenibilidad. Mejora la calidad interna del sistema

RNF04 (Tiempo de Respuesta): Las operaciones críticas de cara al usuario (como "Cotizar Envío" o "Aceptar Viaje") deben procesarse y mostrar una respuesta en pantalla en un tiempo máximo de 4 segundos bajo condiciones de red estables. Una latencia mayor a dos segundos genera frustración en el usuario y aumenta significativamente la tasa de abandono de la transacción.

RNF05 (Seguridad y Autenticación): La gestión de identidades y validación de credenciales debe delegarse a una API de autenticación de terceros (Clerk). La autorización en el backend propio se gestionará mediante tokens seguros (JWT) provistos por dicha API, y todo el tráfico viajará cifrado por HTTPS. Delegar la seguridad crítica a especialistas reduce el riesgo de filtraciones.

RNF06 (Tolerancia a red móvil): La interfaz móvil del Transportista debe estar optimizada para operar en redes de datos celulares (3G/4G) y contar con mecanismos de reconexión automática que eviten la pérdida del estado del viaje ante micro-cortes de señal. El transportista opera en movimiento constante, por lo que su conectividad es inherentemente inestable. Sin estos mecanismos de tolerancia a fallos, el seguimiento en vivo se corrompería constantemente.

1.4 Casos de Uso – Historias de Usuarios

CU-01: Cotizar Envío

CU-01	**Cotizar Envío**
Descripción	El remitente ingresa origen, destino y tipo de paquete para que el sistema calcule y asigna un precio estimado al envío en tiempo real, antes de confirmar la operación.
Actores	Remitente (actor principal). API de Mapas API Meteorológica
Precondiciones	1. Remitente autenticado. 2. API de Mapas disponible. 3. API Meteorológica disponible.
Postcondiciones	1. Sistema asigna costo de envío. 2. Se habilita la opción "Publicar/Solicitar Envío" (CU-02). 3. Se almacena Envio.condicion_climatica.
Secuencia	1. El remitente selecciona "Cotizar Envío" -> Sistema solicita datos. 2. Remitente ingresa datos -> Sistema valida direcciones. (2.1 Si es inválido, solicitar corrección). 3. Sistema consulta clima y registra resultado. (3.1 Si es adverso, aplica recargo RN07).

	4. Sistema calcula costo (Distancia + Multiplicador Categoría RN06 + Recargo RN07). 5. Sistema calcula tiempo estimado. 6. El sistema muestra precio, tiempo y habilita "Solicitar Envío".
Excepciones	E1. Dirección inválida/fuera cobertura: Informar y bloquear. E2. API Mapas no disponible: Informar y cancelar. E3. API Meteorológica no disponible: Calcular sin recargo e informar.
Rendimiento	Pasos 2-6 en máximo 4 segundos.
Frecuencia	Muy alta (50-200 cotizaciones/día).
Importancia	Vital
Urgencia	Inmediata
Comentarios	Cotización con vigencia limitada. El precio se reutiliza en CU-02.

CU-02: Publicar Envío / Solicitar Envío

CU-02	**Publicar Envío / Solicitar Envío**
Descripción	El Remitente confirma la solicitud de un envío, selecciona el método de pago y publica el viaje para que quede visible a los Transportistas que cumplan los requisitos.
Actores	Remitente (actor principal). API Financiera (actor secundario).
Precondiciones	1. El remitente cotizó previamente (CU-01) y no expiró. 2. El remitente tiene medio de pago válido.
Postcondiciones	1. Viaje publicado. 2. Envío en estado 'Buscando'. 3. Transacción creada (estado 'Retenido'). 4. Alerta a Transportistas.
Secuencia	1. Remitente selecciona método de pago y solicita envío -> Sistema verifica vigencia de cotización. 2. Sistema procesa pago. (2.1 Si es rechazado, error). 3. Sistema guarda Envío, crea Transacción y alerta Transportistas. 4. Sistema muestra resumen.
Excepciones	E1. Cotización expirada: redirigir a CU-01. E2. Pago rechazado: error y pedir nuevo método. E3. API Financiera no disponible: informar y cancelar.
Rendimiento	Pasos 1-3 en máximo 7 segundos.
Frecuencia	Alta (30-150 pubs/día).
Importancia	Vital

Urgencia	Inmediata
Comentarios	Dinero retenido en estado 'Retenido' hasta que se marque como 'Entregado'.

CU-03: Trackear Envío

CU-03	**Trackear Envío**
Descripción	El Remitente consulta el estado actual y la ubicación en tiempo real de su envío, visualizando en un mapa al Transportista.
Actores	Remitente (actor principal). API de Mapas (actor secundario).
Precondiciones	1. Envío confirmado. 2. Remitente autenticado.
Postcondiciones	Visualización de estado y, si está 'En Camino', ubicación en tiempo real.
Secuencia	1. Remitente selecciona 'Trackear'. 1.1 Si estado es 'Buscando'/'Aceptado', solo estado textual. 1.2 Si 'Entregado', mostrar estado final. 2. Sistema muestra mapa y ubicación. 3. Sistema actualiza posición cada 10s mientras esté 'En Camino'.
Excepciones	E1. Sin señal GPS: mostrar última ubicación conocida y aviso. E2. API Mapas no disponible: mostrar estado textual sin mapa.
Rendimiento	Mostrar mapa en máx 3 segundos. Actualización cada 10s.
Frecuencia	Moderada/Alta (1-3 consultas por envío).
Importancia	Vital
Urgencia	Inmediata
Comentarios	Mapa solo habilitado en estado 'En Camino'.

CU-04: Calificar Servicio

CU-04	**Calificar Servicio**
Descripción	El Remitente califica al Transportista tras la entrega. Si es baja, sugiere soporte.
Actores	Remitente (actor principal).
Precondiciones	1. Envío en estado 'Entregado'. 2. Remitente autenticado.
Postcondiciones	Envío puntuado (1-5 estrellas). Registro en Reseña creado. Actualización de Transportista.puntuacion_promedio.

Secuencia	1. El remitente selecciona 'Calificar'. 2. Remitente ingresa puntaje y comentario. (2.1 Si <= 2 estrellas, sugerir abrir un ticket de soporte, RF05). 3. El Sistema agradece.
Excepciones	E1. Envío no entregado: error. E2. Ya calificado: error.
Rendimiento	Mensaje de agradecimiento en < 1 seg.
Frecuencia	Moderada (40-70% de envíos).
Importancia	Vital
Urgencia	Inmediata
Comentarios	Escala 1-5 estricta.

CU-05: Explorar Paquetes

CU-05	**Explorar Paquetes**
Descripción	El Transportista visualiza envíos publicados en su radio y compatibles con su vehículo.
Actores	Transportista (actor principal). API Mapas (actor secundario).
Precondiciones	Autenticado, vehículo activo, disponibilidad activada, GPS activo, existen envíos 'Buscando'.
Postcondiciones	Visualización de listado disponible.
Secuencia	1. Transportista accede a explorar. 2. Sistema filtra envíos por radio y capacidad. 3. Sistema muestra listado (destacando Zonas Calientes si aplica).
Excepciones	E1. GPS desactivado: pausar recepción. E2. API Geolocalización no disponible: informar.
Rendimiento	Cargar listado en máx 2 segundos.
Frecuencia	5-20 consultas por jornada por transportista.
Importancia	Vital
Urgencia	Inmediatamente
Comentarios	Filtro de capacidad es estricto.

CU-06: Aceptar Viaje

CU-06	**Aceptar Viaje**

Descripción	El Transportista confirma la aceptación de un envío, vinculándolo a su perfil.
Actores	Transportista (actor principal).
Precondiciones	Autenticado, disponible, GPS activo, envío en 'Buscando', capacidad compatible.
Postcondiciones	Transportista vinculado. Envío pasa a 'Aceptado'. Notificación al Remitente.
Secuencia	1. Transportista solicita aceptar. 1.1 Si ya fue tomado, informar. 2. Transportista confirma. 3. El sistema vincula y notifica al remitente. 4. Sistema muestra pantalla de navegación.
Excepciones	E1. Capacidad excedida: rechazar.
Rendimiento	Procesar en máx 2 segundos.
Frecuencia	3-10 viajes por jornada.
Importancia	Vital
Urgencia	Inmediatamente
Comentarios	Asignación en tiempo real; control de concurrencia necesario.

CU-07: Actualizar Estado del Viaje

CU-07	**Actualizar Estado del Viaje**
Descripción	El Transportista marca 'Retirado' (En Camino) y luego 'Entregado', completando el ciclo.
Actores	Transportista (actor principal). API Financiera (actor secundario).
Precondiciones	Autenticado, estado 'Aceptado'/'En Camino', GPS activo.
Postcondiciones	Si 'Retirado': estado 'En Camino'. Si 'Entregado': estado 'Entregado', pago liberado.
Secuencia	1. Transportista marca 'Retirado'. 1.1 Si ya tiene otro viaje, bloquear. 2. Sistema actualiza a 'En Camino'. 3. Transportista marca 'Entregado'. 4. Sistema actualiza a 'Entregado', libera pago y notifica.
Excepciones	E1. Transportista con otro viaje en tránsito: bloquear. E2. Pérdida GPS: pausar transmisión.
Rendimiento	Procesar cambios en máx 2 segundos.
Frecuencia	2 actualizaciones por viaje.

Importancia	Vital
Urgencia	Inmediata
Comentarios	Pago retenido -> Liberado -> Liquidado.

CU-08: Zonas Calientes

CU-08	**Zonas Calientes**
Descripción	El transportista visualiza zonas de alta demanda para maximizar ganancias.
Actores	Transportista (actor principal).
Precondiciones	Autenticado, disponible, proceso batch nocturno ejecutado, franja horaria activa.
Postcondiciones	Visualización de zonas y multiplicadores.
Secuencia	1. Transportista accede al mapa. 2. Sistema muestra zonas destacadas con multiplicadores. 3. Transportista consulta detalles.
Excepciones	E1. Batch fallido: informar no disponible.
Rendimiento	Cargar mapa en máx 2 segundos.
Frecuencia	1-5 veces por jornada.
Importancia	Importante
Urgencia	Hay presión
Comentarios	Definidas diariamente según historial.

CU-09: Resolución de Conflictos

CU-09	**Resolución de Conflictos**
Descripción	El agente de soporte gestiona y resuelve tickets de soporte.
Actores	Agente de soporte tecnico (actor principal) Remitente (actor secundario).
Precondiciones	Agente de soporte técnico autenticado, tickets pendientes existen, ticket para envío 'Aceptado' o superior.
Postcondiciones	Ticket marcado 'Resuelto'. Remitente notificado.
Secuencia	1. Agente de soporte accede a cola. 2. Agente de soporte toma ticket. 3. Agente de soporte analiza. 4. Agente de soporte redacta conclusión. 5. Agente de soporte confirma resolución.

Excepciones	E1. Cola vacía: informar.
Rendimiento	Carga información en máx 2 segundos.
Frecuencia	5-20 tickets por jornada.
Importancia	Vital
Urgencia	Inmediatamente
Comentarios	Solo resolución técnica.

CU-10: Panel de Métricas

CU-10	**Panel de Métricas**
Descripción	Agente de soporte visualiza estadísticas generales generadas por batch nocturno.
Actores	Agente de soporte(actor principal) Sistema Batch (actor secundario)
Precondiciones	Agente de soporte autenticado, proceso batch ejecutado.
Postcondiciones	Visualización de estadísticas.
Secuencia	1. Agente de soporte accede. 2. Sistema muestra métricas (envíos completados, ganancia). (2.1 Si batch no ejecutó, informar). 3. Agente de soporte consulta.
Excepciones	E1. Batch no ejecutó: mostrar datos anteriores e informar.
Rendimiento	Carga en máx 2 segundos.
Frecuencia	1-3 veces por jornada.
Importancia	Importante
Urgencia	Puede esperar
Comentarios	Actualización diaria.

CU-11: Consultar Historial de Envíos

CU-11	**Consultar Historial de Envíos**
Descripción	El remitente revisa sus envíos realizados.
Actores	Remitente (actor principal).
Precondiciones	Autenticado, al menos un envío registrado.
Postcondiciones	Visualización del historial.

Secuencia	1. Remitente accede. 2. Sistema muestra listado ordenado.

	3. Remitente selecciona detalle.
Excepciones	E1. Error interno: informar.
Rendimiento	Carga en máx 2 segundos.
Frecuencia	1-3 veces por semana.
Importancia	Importante
Urgencia	Hay presión
Comentarios	Incluye envíos en todos los estados.

CU-12: Consultar Historial de Tickets

CU-12	**Consultar Historial de Tickets**
Descripción	El Remitente revisa el estado de sus reclamos.
Actores	Remitente (actor principal).
Precondiciones	Autenticado, al menos un ticket generado.
Postcondiciones	Visualización del historial.
Secuencia	1. Remitente accede. 2. Sistema muestra listado. 3. Remitente selecciona detalle.
Excepciones	E1. Error interno: informar.
Rendimiento	Carga en máx 2 segundos.
Frecuencia	1 por semana.
Importancia	Importante
Urgencia	Hay presión
Comentarios	Estados: Pendiente, En Proceso, Resuelto.

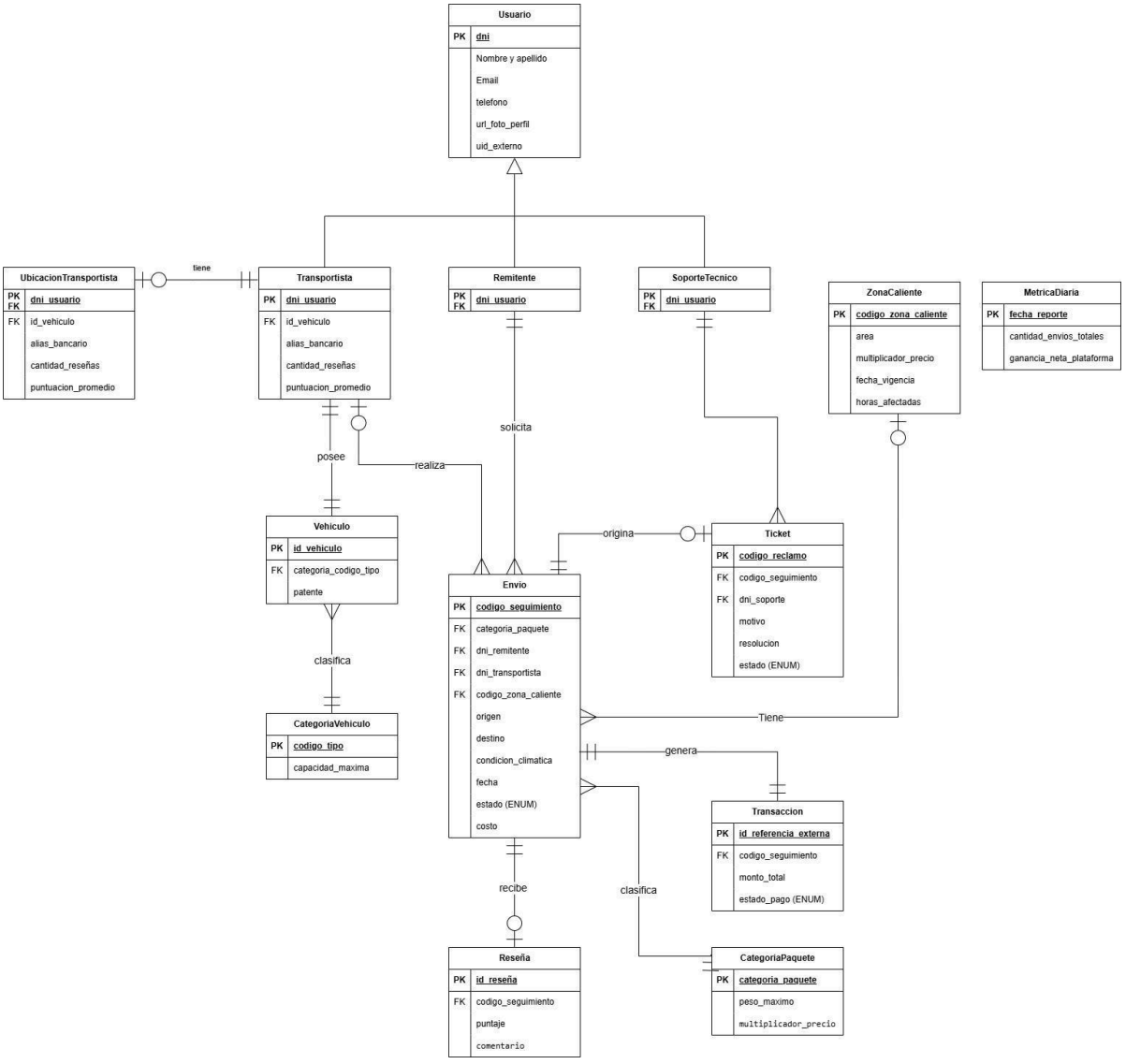
CU-13: Consultar Historial de Viajes

CU-13	**Consultar Historial de Viajes**
Descripción	El Transportista revisa sus servicios completos y ganancias.
Actores	Transportista (actor principal).
Precondiciones	Autenticado, al menos un viaje completado.

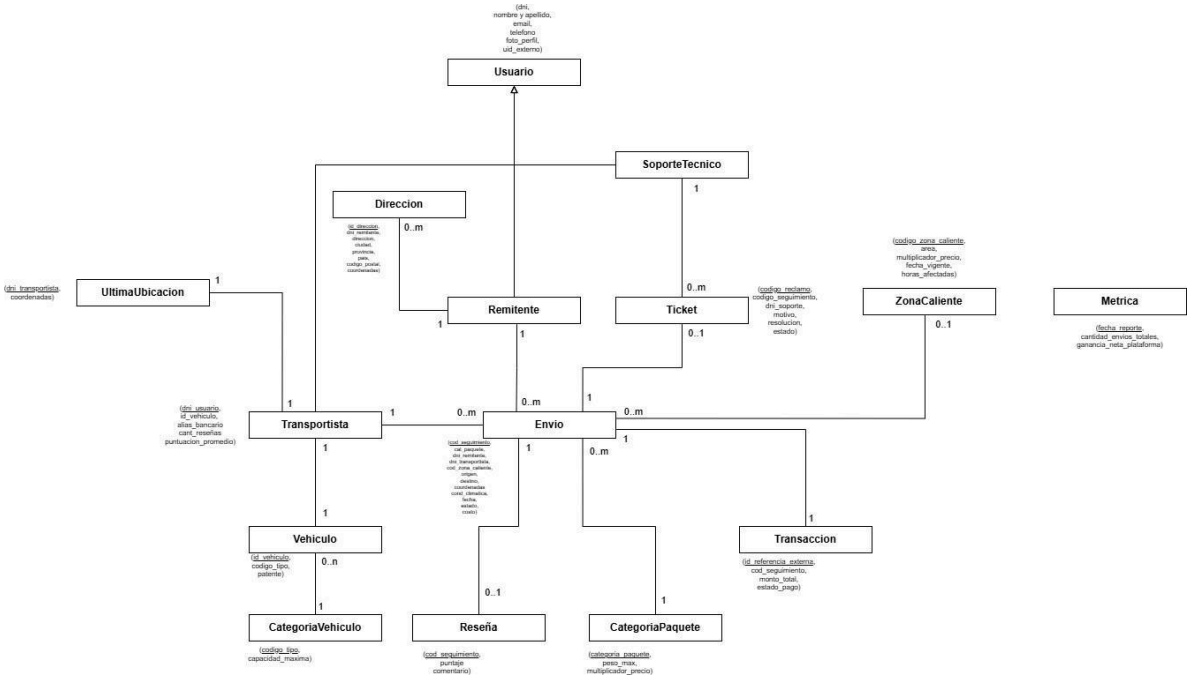
Postcondiciones	Visualización de listado y ganancias.
Secuencia	1. Transportista accede. 2. Sistema muestra listado y total acumulado.

	3. Transportista selecciona detalle.
Excepciones	E1. Error interno: informar.
Rendimiento	Carga en máx 2 segundos.
Frecuencia	1-2 por día.
Importancia	Importante
Urgencia	Hay presión
Comentarios	Incluye bonificaciones por zonas.

1.5 Modelo de Entidad Relación



1.6 Modelo Conceptual de Datos consistente con el Modelo de Entidad Relación



2. Diagramas de Arquitectura

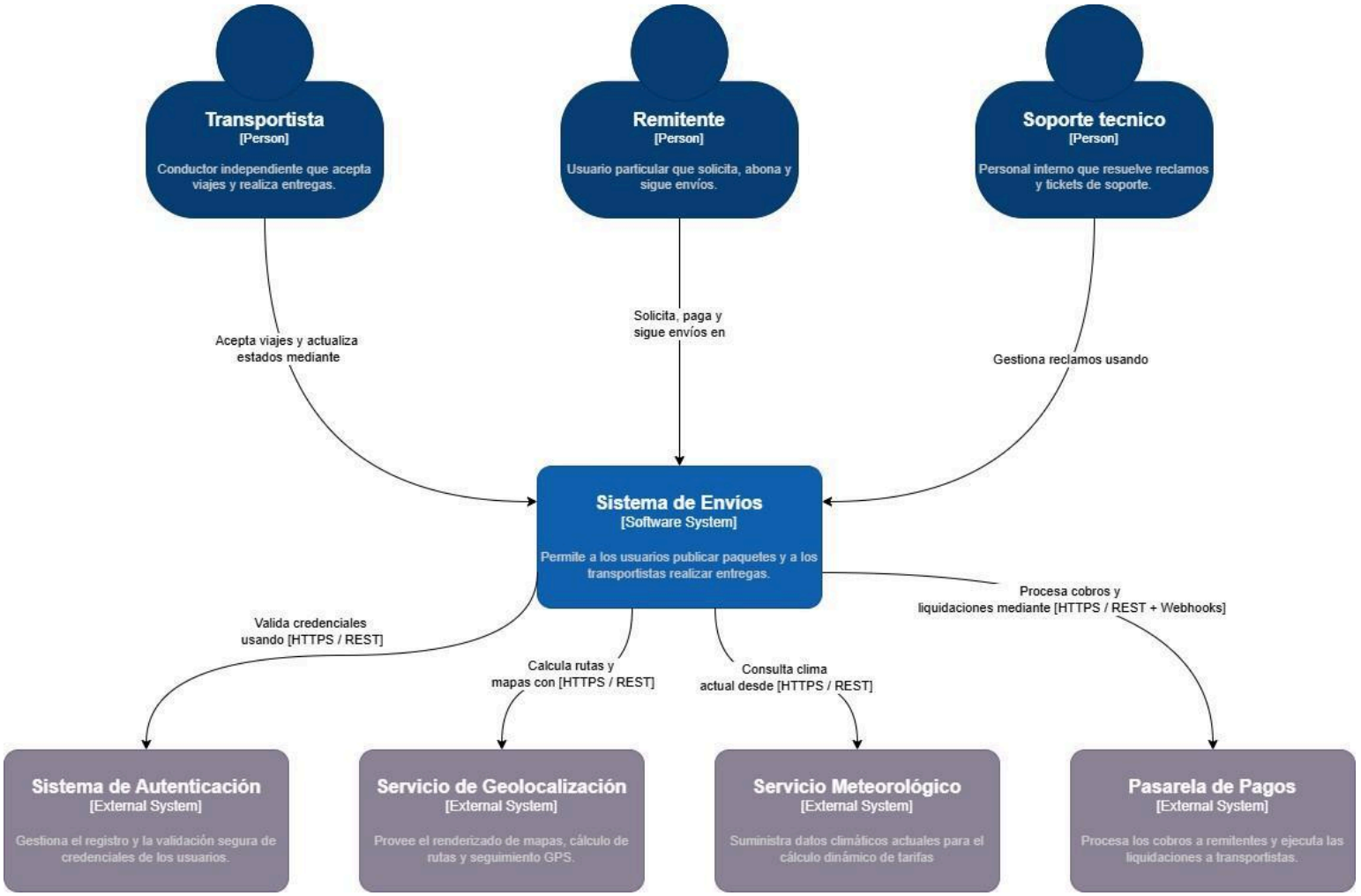
Esta sección corresponde a la documentación arquitectónica del sistema mediante el modelo C4, que describe la arquitectura en niveles de abstracción progresivos: desde la visión de contexto hasta la separación interna de responsabilidades dentro del backend.

Los entregables producidos en esta instancia son:

- Diagrama de Contexto (Nivel 1 C4): define los límites del sistema y sus interacciones con actores y sistemas externos.
- Diagrama de Contenedores (Nivel 2 C4): especifica la arquitectura distribuida con frontend, backend y base de datos.
- Diagrama de Componentes (Nivel 3 C4): detalla la separación de responsabilidades interna del contenedor de backend.

2.1. Diagrama de Contexto

El diagrama de contexto representa al sistema como una caja negra, describiendo únicamente sus interacciones con el entorno externo. No expone detalles internos de implementación, sino que define los límites del sistema y los canales de comunicación con cada actor y sistema externo.



2.1.1 Flujos principales identificados

Flujo 1: Remitente → Ingresa datos para publicar envíos, rastrear paquetes, calificar transportistas o crear reclamos de soporte → Sistema.

Flujo 2: Transportista → Envía solicitudes para buscar y aceptar viajes, registrar los datos de su vehículo y actualizar el estado físico de las entregas → Sistema.

Flujo 3: Soporte Técnico → Envía las conclusiones y resoluciones técnicas para cerrar los reclamos originados por los usuarios, consulta métricas generales del negocio → Sistema.

Flujo 4: Sistema → Envía las credenciales y datos de registro de los usuarios para solicitar su validación → Clerk.

Flujo 5: Sistema → Envía la solicitud para procesar el cobro de un envío o para transferirle las ganancias consolidadas a un transportista → Mercado Pago.

Flujo 6: Sistema → Envía direcciones postales para verificar que existan, o coordenadas para solicitar el cálculo de la mejor ruta → Mapbox.

Flujo 7: Sistema → Envía las coordenadas del lugar donde se va a retirar un paquete para consultar el pronóstico del tiempo → Open-Meteo.

2.1.2 Decisiones de diseño en el nivel de contexto

Decisión 1: Delegación de la Autenticación y Gestión de Identidades (Clerk como sistema externo) Se decidió ubicar la gestión de credenciales y perfiles de seguridad fuera de los límites del sistema principal, delegando íntegramente a Clerk.

- Justificación: La gestión de contraseñas, encriptación y control de sesiones es compleja y no forma parte del "core" de negocio de una plataforma logística. Mantener esto fuera del límite de nuestro sistema reduce drásticamente el riesgo de filtraciones de datos sensibles y nos permite asegurar el acceso mediante estándares de la industria sin tener que desarrollar ni mantener esa infraestructura de seguridad internamente.

Decisión 2: Tercerización del procesamiento financiero (Mercado Pago como pasarela) Se decidió que la plataforma no almacenará dinero real ni funcionará como una billetera virtual independiente, sino que dependerá de Mercado Pago para procesar los cobros a los remitentes y las liquidaciones a los transportistas.

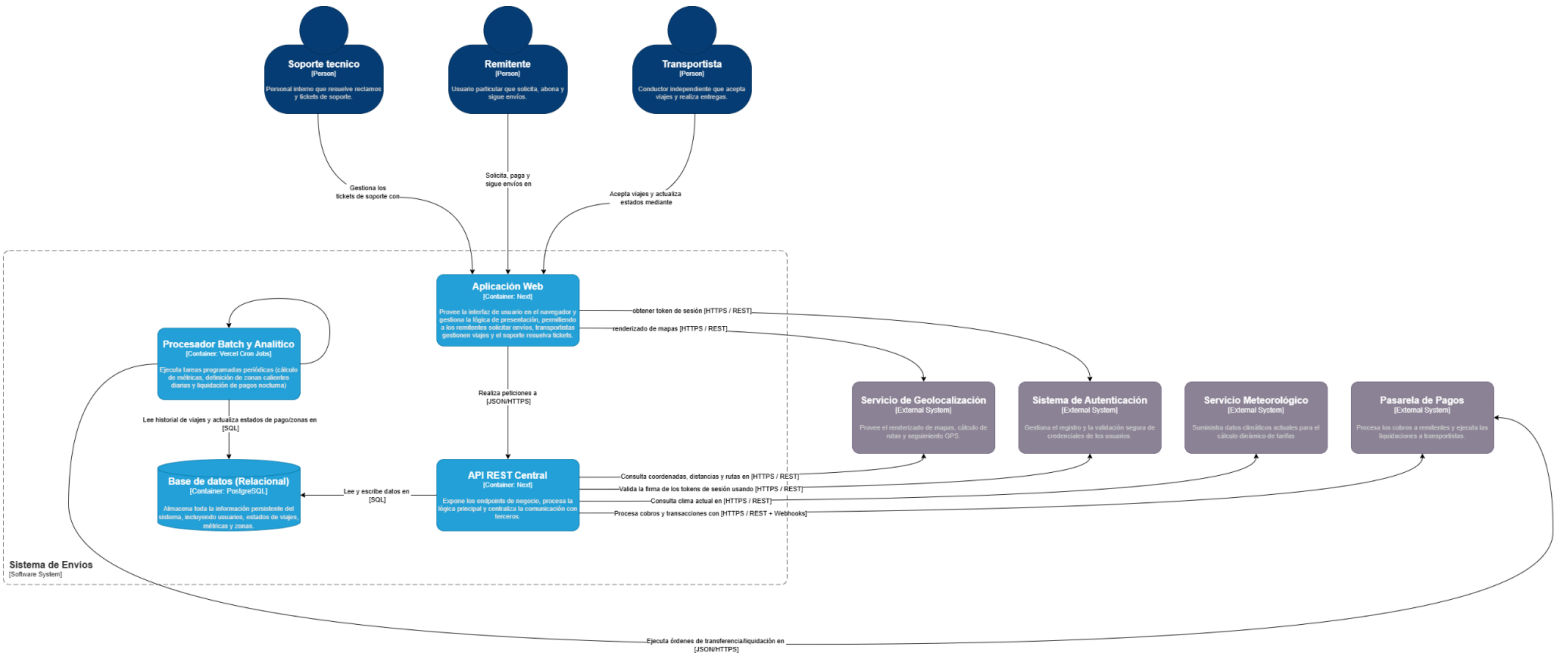
- Justificación: El sistema requiere una máquina de estados compleja para el dinero ("Retenido", "Liberado", "Liquidado"). Asumir el rol de procesador de pagos implicaría enormes responsabilidades legales, de cumplimiento y altos estándares de seguridad. Al dejar el procesamiento del dinero real fuera de nuestro límite, nuestro sistema actúa únicamente como un "orquestador" que guarda referencias a transacciones externas, reduciendo el riesgo legal y la carga transaccional.

Decisión 3: Restricción del límite de uso del módulo de Soporte. Se decidió dejar fuera del alcance de esta versión la posibilidad de que los Transportistas inicien tickets de soporte, limitando esta funcionalidad exclusivamente al actor "Remitente".

- Justificación: Permitir la apertura de disputas bilaterales (ej. Un conductor reportando que el remitente no está en el domicilio) aumentaría significativamente la complejidad de los estados del envío. Para mantener un alcance manejable que permita lanzar esta primera versión del sistema a tiempo, se trazó un límite estricto de negocio donde solo el consumidor final (Remitente) puede auditar el servicio.

2.2 Diagrama de Contenedores

El diagrama de contenedores descompone el interior del sistema en unidades desplegables de forma independiente. Cada contenedor representa una tecnología concreta y una responsabilidad bien delimitada.



2.2.1 Contenedores del sistema

App web unificada — Next.js (Framework de React)

Qué hace y por qué**:** Es la capa de presentación del sistema y el único punto de interacción directo para los tres actores humanos. Se encarga de capturar las acciones del usuario y renderizar de forma interactiva su vista correspondiente. Se seleccionó Next.js debido a su capacidad para construir interfaces dinámicas de alto rendimiento. Además, su elección responde a una estrategia de optimización de tiempos de desarrollo y reutilización de competencias técnicas previas del equipo, alineando el proyecto con las tecnologías trabajadas simultáneamente en el ámbito académico. Su naturaleza *web-responsive* permite resolver las necesidades de acceso desde múltiples dispositivos (móviles y de escritorio) mediante un único despliegue, evitando la sobrecarga y complejidad operativa de desarrollar aplicaciones nativas independientes para cada plataforma.
Se comunica con:

- Envía peticiones a la API REST Central mediante solicitudes seguras HTTPS siguiendo el estándar REST/JSON. Para el seguimiento en tiempo real, utiliza un mecanismo de "Polling", consultando periódicamente al backend por la ubicación actualizada del envío.
- Se comunica con el proveedor externo Clerk vía HTTPS para enviar las credenciales y recibir el token seguro de sesión (JWT). Además del JWT, obtiene el rol del usuario mediante metadatos.
- Se conecta con la API externa de Mapbox vía HTTPS para renderizar los mapas interactivos en la página.

API REST Central — Next.js (React)

Es el núcleo del sistema. Aloja la lógica transaccional, las políticas de seguridad y las reglas de negocio de Packeteer. Recibe peticiones del frontend, valida los payloads y orquesta las consultas a la base de datos.
Se comunica con:

- Recibe peticiones del Frontend vía HTTPS (REST/JSON), incluyendo las consultas repetitivas de ubicación (*polling*).
- Se comunica con la Base de Datos Relacional mediante el protocolo nativo del motor de base de datos sobre una conexión TCP/IP segura.
- Consume las APIs externas de Mercado Pago (pagos), Open-Meteo (clima), Mapbox (rutas) y Clerk (autenticación) utilizando llamadas HTTPS (REST/JSON).

Base de datos — PostgreSQL

Se encarga del almacenamiento persistente, seguro y estructurado de toda la información del dominio. Cuenta con restricciones que aseguran que los estados de los envíos, pagos y tickets cumplan las reglas de negocio en la capa de datos. Se eligió **PostgreSQL** por ser un motor relacional extremadamente robusto, con pleno soporte para transacciones ACID (vital para manejar los bloqueos y evitar que dos transportistas acepten el mismo viaje simultáneamente) y por su compatibilidad nativa con herramientas de mapeo como Prisma.

Se comunica con:

- Recibe de forma exclusiva solicitudes de lectura y escritura (sentencias SQL generadas por Prisma) provenientes del Backend y del Procesador de Tareas Batch a través de conexiones TCP/IP. Está completamente aislada del acceso público de internet.

Procesador Batch y Analítico — Vercel Cron Jobs

Ejecuta procesos analíticos de una gran cantidad de datos en segundo plano de forma diferida. Sus responsabilidades incluyen: correr el algoritmo analítico de "Zonas Calientes" basándose en el historial de viajes, y consolidar las métricas de ganancias. Se eligieron los **Vercel Cron Jobs** porque se integran de forma nativa con el ecosistema de Next.js, permitiendo disparar tareas programadas en horarios de baja concurrencia (ej. madrugada) sin necesidad de mantener ni pagar un servidor encendido 24/7 de forma exclusiva para esto.

Se comunica con:

- Llama a *endpoints* específicos y protegidos del Backend vía HTTPS para gatillar la lectura de grandes volúmenes de datos en la base de datos.
- *(Nota para la próxima etapa)*: Se comunicará de manera externa con Mercado Pago vía HTTPS para dar la orden de dispersión de fondos masiva hacia los bancos de los transportistas.

2.2.2 Decisiones arquitectónicas

Organización de servicios

Se optó por centralizar el sistema en un único repositorio y despliegue unificado conformando un monolito. Sin embargo, internamente se aplica una separación lógica de responsabilidades en capas. El Frontend actúa como una capa de presentación aislada que se comunica exclusivamente vía HTTP/REST con el Backend. El Backend, por su parte, encapsula la lógica de negocio y el acceso a datos siguiendo el patrón MVC. Esta arquitectura nos otorga las ventajas de un desarrollo ágil, un despliegue simplificado y tipado compartido, evitando la sobrecarga operativa y de red que implicaría mantener microservicios separados para el alcance actual del proyecto. Al estar acoplados en el despliegue, asumimos que si un componente específico (ej. módulo de envios) demanda altos recursos, nos veremos obligados a escalar el servidor y la aplicación completa, ya que no es posible escalar el backend independientemente del frontend.

Estrategia de persistencia

Elegimos una Base de datos 100% Relacional sin capa de caché intermedia. El núcleo del negocio exige consistencia transaccional estricta . Entidades como Envio, Transacción y Transportista están altamente interrelacionadas. Postgres nos permite evitar condiciones de carrera críticas (ej. evitar que dos conductores acepten el mismo paquete al mismo milisegundo).

Manejo de datos de alta frecuencia (GPS) y Caché: Conscientes del volumen de lecturas y escrituras que generan las actualizaciones de ubicación en tiempo real, se evaluó implementar una base de datos en memoria (como Redis/Upstash) como caché temporal. Sin embargo, se decidió descartarla para esta etapa y escribir las coordenadas GPS directamente en PostgreSQL.

Limitaciones asumidas: Se priorizó mantener una única fuente de verdad y reducir la complejidad operativa (evitando lidiar con la invalidación de caché). El *trade-off* asumido es que el motor de PostgreSQL absorberá una carga de transacciones significativamente mayor, lo cual podría requerir escalar el hardware de la base de datos antes de lo previsto si la concurrencia de viajes aumenta drásticamente.

Mecanismo de comunicación entre componentes

Comunicación Síncrona (REST/HTTPS) combinada con Polling, y tareas Batch asíncronas.

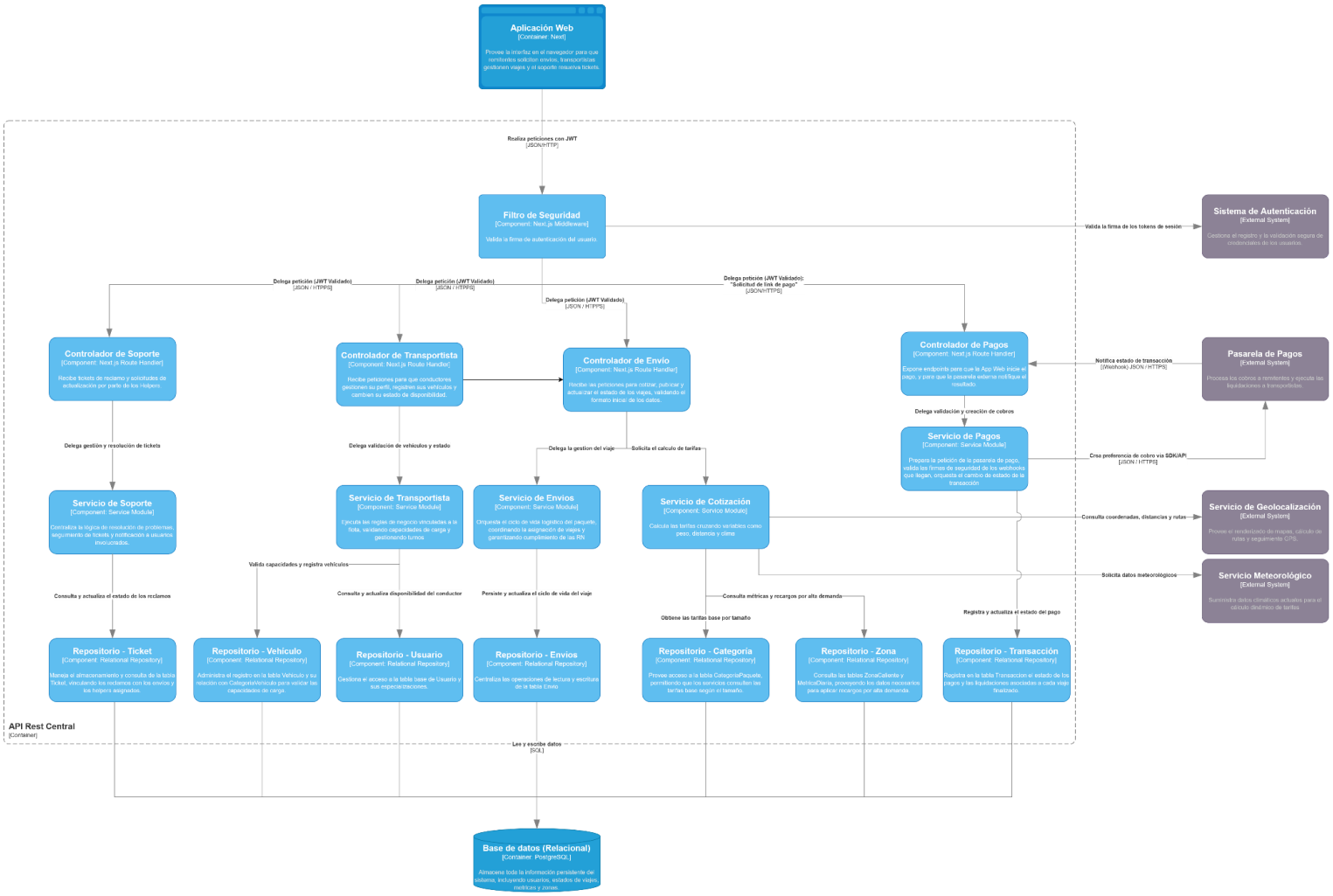
Todo el flujo de llamadas entre el frontend, el Backend y las integraciones de terceros (Clerk, Mercado Pago, etc…) utiliza peticiones síncronas REST. Es el estándar más seguro, maduro y soportado de manera nativa por Next.js.

Seguimiento GPS: Para la comunicación en tiempo real, se descartó el uso de WebSockets (orientado a eventos bidireccionales) debido a que los entornos *serverless* (como Vercel) no mantienen conexiones continuas abiertas. Para sortear esto, se implementó un mecanismo de **Polling**, donde el cliente realiza peticiones síncronas (REST) cada cierta cantidad de segundos para pedir la nueva ubicación.

Asincronismo: Los procesos pesados o que no requieren respuesta inmediata para el usuario (como calcular "Zonas Calientes" o ejecutar dispersiones de dinero) no se comunican en el flujo normal, sino que se delegan de forma asíncrona a un planificador de tareas diferidas (Vercel Cron Jobs).

2.3. Diagrama de Componentes

El diagrama de componentes detalla la organización interna del contenedor de backend, identificando los módulos con responsabilidades bien definidas y las relaciones entre ellos.



2.3.1 Arquitectura interna adoptada

El contenedor de la API REST Central (desarrollado con Next.js) está estructurado internamente siguiendo una separación lógica de responsabilidades basada en el patrón de arquitectura en capas (Layered Architecture). Esta decisión de diseño garantiza la modularidad, facilita el mantenimiento y permite separar claramente la lógica de enrutamiento de las reglas de negocio y el acceso a datos.

Las capas definidas son:

- Filtro de Seguridad (Middleware): Actúa como la primera línea de defensa, interceptando todas las peticiones entrantes para validar la autenticación (tokens JWT) antes de que alcancen los controladores.
- Controladores (Next.js Route Handlers): Son los puntos de entrada de la API. Se encargan exclusivamente de recibir las peticiones HTTP, validar la estructura inicial de los datos (payloads) y delegar la ejecución a la capa de servicios, devolviendo luego la respuesta HTTP correspondiente.

- Servicios (Service Modules): Encapsulan el nucleo del sistema. Aquí reside toda la lógica de negocio, las reglas de dominio y la orquestación de llamadas a sistemas externos (APIs de terceros) y a los repositorios de datos.
- Repositorios (Relational Repositories): Forman la capa de abstracción de datos. Son los únicos componentes autorizados a ejecutar operaciones de lectura y escritura directamente contra la Base de Datos Relacional (PostgreSQL), ocultando los detalles de las consultas SQL al resto del sistema.

2.3.2 Descripción de componentes

Capa de seguridad e Interceptores

- Filtro de Seguridad: Valida la firma de autenticación del usuario. Interactúa con el Sistema de Autenticación (Clerk) externo para verificar el token JWT y, en caso de ser válido, delega la petición hacia los controladores internos.

Capa de Controladores

- Controlador de Soporte: Recibe tickets de reclamo y solicitudes de actualización por parte de los agentes de soporte tecnico. Delega la gestión y resolución de estos al Servicio de Soporte.
- Controlador de Transportista: Recibe peticiones para que los conductores gestionen su perfil, registren sus vehículos y cambien su estado de disponibilidad. Delega la validación de estos datos al Servicio de Transportista.
- Controlador de Envio: Recibe las peticiones para cotizar, publicar y actualizar el estado de los viajes, validando el formato inicial de los datos. Delega la gestión principal del viaje al Servicio de Envios y solicita el cálculo de costos al Servicio de Cotización.
- Controlador de Pagos: Espera la respuesta de la App Web para iniciar un pago y recibe el estado de la transacción mediante un Webhook asíncrono desde la Pasarela de Pagos. Delega la validación y creación del cobro al Servicio de Pagos.

Capa de Servicios de Negocio

- Servicio de Soporte: Centraliza la lógica de resolución de problemas, seguimiento de tickets y notificación a los usuarios involucrados. Consulta y actualiza el estado de los reclamos interactuando con el Repositorio - Ticket.
- Servicio de Transportista: Ejecuta las reglas de negocio vinculadas a la flota, validando las capacidades de carga y gestionando turnos. Interactúa con el Repositorio - Vehículo (para validar capacidades) y el Repositorio - Usuario (para actualizar la disponibilidad).
- Servicio de Envios: Orquesta el ciclo de vida logístico del paquete, coordinando la asignación de viajes y garantizando el cumplimiento de las reglas de negocio. Persiste y actualiza los estados interactuando con el Repositorio - Envios.
- Servicio de Cotización: Calcula las tarifas dinámicas cruzando variables como peso, distancia y clima. Para ello, solicita datos externos al Servicio de Geolocalización y al Servicio Meteorológico, y obtiene los parámetros base de negocio interactuando con el Repositorio - Categoria y el Repositorio - Zona.
- Servicio de Pagos: Prepara la petición de la pasarela de pago, encapsulando firmas de seguridad y manejando la interfaz de enlace. Crea preferencias de cobro llamando a la Pasarela de Pagos externa y registra el estado final interactuando con el Repositorio - Transaccion.

Capa de Repositorios / Acceso a datos

(Nota: Todos los componentes de esta capa se comunican directamente de forma síncrona con la Base de datos (Relacional) para ejecutar las lecturas y escrituras de la información persistente del sistema)

- Repositorio - Ticket: Maneja el almacenamiento y consulta de la tabla Ticket, vinculando los reclamos con los envíos correspondientes y los agentes de soporte tecnico asignados.
- Repositorio - Vehiculo: Administra el registro en la tabla Vehículo, cruzando los datos matriculados para establecer las capacidades de carga de cada transporte.
- Repositorio - Usuario: Gestiona el acceso a la tabla base de Usuario y sus especializaciones (roles), permitiendo consultar y actualizar estados de cuentas.
- Repositorio - Envios: Centraliza las operaciones de lectura y escritura exclusivas de la tabla Envío.
- Repositorio - Categoria: Proporciona acceso a la tabla CategoriaPaquete, permitiendo que los servicios consulten las tarifas base según el tamaño del paquete.
- Repositorio - Zona: Consulta las tablas ZonaCaliente e HistorialVisitas, proveyendo los datos necesarios para aplicar recargos dinámicos por alta demanda.
- Repositorio - Transaccion: Registra en la tabla Transaccion el estado financiero de los pagos y las liquidaciones asociadas a cada viaje finalizado.

3. Stack Tecnológico

Next.js (Framework de React) — Capa de Presentación (Frontend) y API REST (Backend): Se utiliza como el framework central que aloja tanto la interfaz de usuario web interactiva como las rutas de la API del backend en un único repositorio (Monolito Full-Stack). En lugar de mantener dos proyectos separados con distintas tecnologías. Next.js permite compartir el tipado estricto entre el cliente y el servidor. Además, facilita la implementación del *Polling* para el seguimiento en tiempo real y está optimizado nativamente para desplegarse en entornos *serverless* como Vercel, reduciendo la carga operativa del equipo.

PostgreSQL — Base de Datos Relacional: Se usa para el almacenamiento persistente y estructurado de todas las entidades del dominio. Como se menciono antes el sistema logístico exige consistencia absoluta.Una base de datos NoSQL no nos brindaría estas garantías de forma nativa. Además, soporta restricciones duras para la máquina de estados.

Prisma ORM y Zod — Acceso a Datos y Validación: Prisma actúa como el intermediario entre la API y PostgreSQL, mientras que Zod valida la estructura de los datos que entran al sistema. Prisma agiliza enormemente el modelado de la base de datos y la generación de consultas SQL seguras (previniendo inyecciones SQL). Por su parte, Zod fue elegido por su integración perfecta con TypeScript, permitiendo rechazar peticiones mal formadas en el backend antes de que toquen la lógica de negocio.

Shadcn/ui y Tailwind CSS — Biblioteca de Componentes y Estilos (Frontend): Se utiliza para construir toda la interfaz de usuario (UI) interactiva de la plataforma, proveyendo la estructura base de formularios, modales, alertas y navegación. Shadcn inyecta el código fuente de los componentes directamente en nuestro repositorio. Esto nos da control absoluto sobre el renderizado y el diseño. Combinado con **Tailwind CSS**, nos permitió acelerar drásticamente la construcción de las pantallas de Packeteer manteniendo una interfaz accesible

4. Patrones de Diseño y Atributos de Calidad

4.1 Patrones aplicados

Patrón 1: UI-Hook-Service (Estructural / Frontend): Aplica en toda la arquitectura de la aplicación web Evita el acoplamiento y el "código espagueti" en el frontend. Separa estrictamente la capa de presentación visual (componentes UI), la gestión del estado y ciclo de vida (Custom Hooks), y la comunicación con el backend (Services). Esto permite que si mañana cambia la URL de la API o la forma de hacer el fetch, solo se modifique el archivo del servicio sin tocar ni romper el diseño de las pantallas.

Patron 2: MVC - Modelo-Vista-Controlador (Estructural / Backend): En la capa del servidor (API REST Central con Next.js y Prisma). Los *Controladores* (Next.js API Routes) reciben las peticiones HTTP y validan los datos con Zod. El *Modelo* (Prisma ORM) se encarga exclusivamente de las reglas de acceso a PostgreSQL. La *Vista* (en esta arquitectura desacoplada) está representada por las respuestas JSON formateadas que el controlador le devuelve al cliente.

Patrón 3: Polling (Comportamiento / Comunicación Frontend-Backend): Se aplica en el flujo de rastreo en vivo del envío entre el dispositivo del remitente y la API REST. Esto Permite simular actualizaciones en tiempo real de la ubicación GPS del Transportista en un entorno de despliegue *serverless* (Vercel), donde no es técnica ni económicamente viable mantener conexiones continuas abiertas (como WebSockets).

4.2 Atributos de calidad

RNF01 - Integración con APIs externas: La arquitectura de la API REST Central fue diseñada explícitamente para comunicarse de forma síncrona mediante HTTPS y JSON con los proveedores elegidos en el Contexto (Mapbox, Open-Meteo, Clerk y Mercado Pago). Para asegurar que el sistema interno no falle si un proveedor externo cambia inesperadamente el formato de sus respuestas, se implementó la librería Zod en el backend. Zod valida y parsea todos los payloads JSON entrantes y salientes, garantizando un contrato estricto de integración.

RNF02 - Integración (Procesamiento analítico diferido): Se extrajo la responsabilidad de los cálculos masivos (como la definición diaria de Zonas Calientes y consolidación de métricas globales) fuera del flujo transaccional principal del monolito. Arquitectónicamente, esto se satisface mediante el uso de un contenedor separado de Procesos Batch (Vercel Cron Jobs), programado para ejecutarse de forma asíncrona y exclusivamente en horarios de baja concurrencia (madrugada), garantizando que los usuarios no sufran latencia por consumo masivo de base de datos durante sus operaciones diurnas.

RNF03 - Patrones de Diseño (Arquitectura de Frontend y Backend): El sistema centraliza su lógica en un Monolito, pero mantiene responsabilidades estrictamente separadas en capas para garantizar escalabilidad y mantenibilidad (Principios SOLID):

- En el Frontend: Se implementa el patrón UI-Hook-Service en Next.js, logrando un alto grado de desacoplamiento que permite crear mocks de datos en la capa de servicios para avanzar con el diseño de las pantallas sin depender de la API finalizada.
- En el Backend: Se adopta el patrón MVC, donde la capa de acceso a datos (Modelo) está gestionada por Prisma ORM, manteniéndola fuertemente tipada (TypeScript) y completamente separada de la lógica de ruteo de la API.

RNF04 - Tiempo de Respuesta (< 4s): La decisión de utilizar un Monolito (alojando front y back en los mismos servidores serverless de Vercel) minimiza los saltos de red (network hops). Además, al optar por consultas transaccionales directas a la base de datos relacional (PostgreSQL) y evitar la sobrecarga de intermediarios de caché para el flujo normal, se redujo la latencia técnica al mínimo indispensable para cumplir con el umbral de los 4 segundos exigidos por el negocio en flujos críticos.

RNF05 - Seguridad y Autenticación: Esta restricción se resuelve directamente a nivel de diagrama de contexto mediante la adopción de Clerk como sistema externo especializado. Nuestro backend no almacena contraseñas ni gestiona sesiones en la base de datos (PostgreSQL); solo confía y verifica criptográficamente los tokens JWT emitidos por Clerk. Esto delega la validación de credenciales y la protección contra filtraciones directamente al proveedor, tal como lo exige el dominio.

RNF06 - Tolerancia a red móvil: Al haber optado por una aplicación web unificada (Next.js) en lugar de una app nativa, se descartó el uso de WebSockets (cuyas conexiones se rompen fácilmente con micro-cortes de señal 3G/4G) a favor de un mecanismo de Polling síncrono (HTTP/REST). Arquitectónicamente, los mecanismos de reconexión automática se satisfacen en la capa Service del Frontend: si el transportista pierde la señal al enviar su actualización de GPS, el cliente web captura el error silenciosamente y reintenta la petición en el próximo ciclo (ej. a los 10 segundos), recuperando el estado del viaje automáticamente cuando vuelve la conexión.

5. Flujos Principales del Sistema

5.1 Flujo de autenticación

Este flujo describe cómo un usuario ingresa a la plataforma y cómo el sistema valida su identidad en cada acción posterior sin comprometer la seguridad.

1. El usuario ingresa sus credenciales (o utiliza un proveedor como Google) en el frontend a través de una interfaz segura provista por clerk.
2. Clerk valida las credenciales y, si son correctas, genera un token JWT firmado criptográficamente que devuelve al cliente.
3. El Frontend almacena este token de forma segura (gestionado por el SDK de Clerk).
4. El usuario navega hacia una ruta protegida o realiza una petición, envía ese JWT. El Middleware de Next.js (Proxy interno) intercepta la solicitud antes de procesar la lógica de negocio o renderizar la vista.
5. El Middleware decodifica el JWT de sesión para extraer de forma segura el identificador único del usuario.
6. Debido a las restricciones de la capa gratuita del proveedor (la cual no permite inyectar metadatos personalizados directamente en el payload del JWT de sesión), el Middleware realiza una llamada saliente vía HTTPS a la API de Clerk utilizando el userId para consultar activamente los datos completos del perfil.
7. Al recibir la respuesta de Clerk, el sistema evalúa la propiedad *public metadata* aplicando un patrón de rol implícito por defecto: si la metadata indica explícitamente los valores de *Transportista* o *Admin (soporte técnico)*, se le asigna dicho rol; en caso de que la metadata se encuentre vacía, el Middleware clasifica automáticamente a la cuenta con el rol de *Remitente* (usuario estándar).
8. Con el rol determinado en tiempo de ejecución, el Middleware valida los permisos para la ruta solicitada: si el usuario cuenta con los privilegios requeridos (ej. un Remitente accediendo a la vista de "Cotizar Envío"), se le permite el acceso; de lo

contrario, se intercepta la petición, bloqueando el paso y redirigiendo a la página principal de su tipo de usuario.

5.2 Flujo de Cotización y Publicación de Envío

Este es el flujo principal del dominio para el actor Remitente. Orquesta múltiples sistemas externos y aplica reglas de negocio de precios dinámicos.

1. Selección y Carga de Direcciones: El Remitente accede a su pantalla principal en la App Web unificada y selecciona la opción "Cotizar Envío". El sistema presenta dos campos de texto (Origen y Destino) que integran el componente SearchBox de Mapbox para asegurar el ingreso de direcciones postales reales. Alternativamente, para el campo de origen, el usuario puede seleccionar una de sus direcciones pre guardadas de su perfil.
2. Visualización en Tiempo Real: Cada vez que el usuario selecciona un punto (origen o destino), el Frontend envía las coordenadas a la API externa de Mapbox para renderizar un marcador interactivo en el mapa. Una vez seleccionados ambos puntos, el SDK de Mapbox calcula y dibuja visualmente en pantalla la ruta óptima de transporte.
3. Parámetros del Paquete: El Remitente selecciona la categoría de tamaño del paquete (S, M o L). El listado de categorías y sus coeficientes base son obtenidos dinámicamente mediante una consulta síncrona a la API REST Central (Backend).
4. Cálculo de Tarifa Dinámica: Al presionar "Cotizar", la API REST Central intercepta la solicitud. El backend realiza una llamada externa a la API de Open-Meteo enviando las coordenadas de origen para registrar la condición climática actual. Con estos datos, el controlador calcula la tarifa final cruzando la distancia (Mapbox), el clima (Open-Meteo) y el multiplicador vigente de "Zonas Calientes" (generado por el Procesador Batch), mostrando el resumen de costo y tiempo estimado en la UI.
5. Persistencia del Borrador: Si el usuario decide avanzar, el estado de la cotización se almacena temporalmente como un objeto DraftEnvio dentro del sessionStorage del navegador para resguardar la información ante posibles recargas accidentales, y se lo redirige a la vista de caja (Checkout).
6. Pasarela de Pagos (Mercado Pago): En la vista de Checkout, entra en juego el custom hook useCheckout, el cual recupera el borrador del sessionStorage. Al presionar el botón "Pagar", el hook invoca de forma asíncrona al checkoutService. Este servicio envía el borrador a la API REST Central, la cual interactúa con Mercado Pago para generar una preferencia transaccional y devuelve un enlace seguro de redirección (Sandbox Link). El Frontend redirige al Remitente hacia el portal externo de pagos.
7. Confirmación Asíncrona (Webhook): Una vez que el Remitente procesa el pago exitosamente dentro de la plataforma de Mercado Pago, dicha pasarela envía de forma asíncrona una notificación Webhook (Petición HTTPS POST) hacia un endpoint dedicado y expuesto en nuestra API REST Central.
8. Publicación del Envío: El backend procesa el Webhook, valida la veracidad del pago contra Mercado Pago y abre una transacción en la Base de Datos (PostgreSQL): persiste la orden con un registro Envio fijando obligatoriamente su campo estado en 'Buscando' (en cumplimiento estricto de la regla de negocio RN01) y guarda la Transaccion financiera con el campo estado_pago en 'Retenido' (en cumplimiento de la regla RN10), quedando visible inmediatamente para los transportistas en zona.
9. Redirección y Tracking: En paralelo, Mercado Pago redirige al Remitente de vuelta a la ruta de retorno de la App Web unificada. El hook useCheckout detecta la confirmación de éxito, ejecuta un sessionStorage.removeItem("draft_envio") para limpiar la memoria caché, despliega una alerta de confirmación en pantalla mediante

componentes de shadcn/ui y redirige al actor a la pantalla principal, donde se inicia el Polling periódico para trackear el paquete.

5.3 Flujo de Liquidación de Pagos Nocturna

Este flujo asíncrono se encarga de transferir el dinero retenido a las cuentas bancarias de los transportistas por los viajes completados. Orquesta la comunicación con la pasarela de pagos externa y aplica mecanismos de resiliencia ante fallos temporales.

1. Disparador Asíncrono (Cron Job): A las 03:00 AM, el planificador de tareas externo (Vercel Cron) ejecuta una petición HTTP POST segura (validada mediante un secreto CRON_SECRET) hacia el endpoint /api/jobs/liquidacion de la API REST Central.
2. Exclusión Mutua (Mecanismo Técnico): El caso de uso intenta adquirir un candado distribuido (Distributed Lock) en la base de datos PostgreSQL. Esto garantiza que si múltiples instancias del servidor intentan ejecutar el proceso en simultáneo, solo una proceda, evitando dobles pagos.
3. Identificación de Lote: El sistema consulta al repositorio y agrupa todas las transacciones financieras que se encuentren habilitadas para ser liquidadas.
4. Desglose y Reserva de Estado: Por cada transacción, el sistema calcula la comisión de la plataforma y el monto final del transportista. Utilizando el patrón Unit of Work, se actualiza la transacción en PostgreSQL al estado temporal "Liberado" antes de contactar al proveedor externo.
5. Comunicación con Sistema Externo: El backend envía una solicitud de transferencia vía API a la Pasarela de Pagos (Mercado Pago). Para evitar transferencias duplicadas en caso de interrupciones de red, la solicitud se envía adjuntando una *idempotencyKey* única.
6. Tolerancia a Fallos (Backoff): Si la API de pagos devuelve un error temporal (ej. timeout), el sistema aplica un mecanismo automático de reintentos con espera exponencial (Exponential Backoff), pausando la ejecución progresivamente (1s, 2s, 4s) antes de reintentar.
7. Resolución y Consolidación:
    - Exitosa: Si la transferencia se concreta, se utiliza Unit of Work para marcar la transacción como "Liquidada" y guardar el comprobante externo.
    - Rechazada: Si el proveedor la rechaza por un error permanente (ej. cuenta destino inválida), la transacción pasa a estado "Revisión Manual" para la intervención de soporte.

6. Conclusión

6.1 Análisis de la arquitectura frente al dominio

El diseño estructural de Packeteer fue concebido para alinearse con la naturaleza dinámica, transaccional y geoespacial que define a una plataforma de logística urbana colaborativa. El núcleo del negocio demanda una consistencia inmediata en operaciones de alta criticidad, como asegurar que no existan colisiones de concurrencia cuando dos transportistas intentan aceptar una misma oferta. Un modelo de *monolito modular* estructurado en capas, con el respaldo de un motor relacional como **PostgreSQL**, satisface estos requerimientos al habilitar bloqueos transaccionales estrictos (ACID) que garantizan la integridad de los datos.

Adicionalmente, el sistema responde a la necesidad de interactuar con variables financieras y meteorológicas delegando la infraestructura periférica a proveedores especializados (Clerk, Mercado Pago y Open-Meteo), lo cual permite que el backend central se focalice en la orquestación del servicio y el procesamiento de las reglas de negocio.

6.2 Compromisos de diseño y trade-offs

A fin de agilizar el desarrollo bajo un paradigma *serverless* (Vercel) y considerando los recursos disponibles, el equipo adoptó tres decisiones arquitectónicas con sus respectivas limitaciones aceptadas:

1. *Estrategia de Polling:* Se priorizó un mecanismo de consultas periódicas vía HTTPS/REST para el rastreo GPS por sobre el uso de WebSockets, asumiendo una mayor carga operativa a cambio de asegurar la estabilidad del flujo dentro de la infraestructura serverless.
2. *Persistencia sin Capa de Caché:* Se simplificó el esquema de almacenamiento descartando motores en memoria como Redis. Se aceptó conscientemente que PostgreSQL absorba el volumen total de lecturas y escrituras dinámicas para mantener una fuente de verdad única y reducir la complejidad técnica inicial.
3. *Estructura Monolítica:* Se privilegió la simplicidad de mantenimiento y el despliegue unificado entre frontend y backend, postergando la capacidad de escalamiento independiente de componentes en favor de una entrega más ágil y un tipado compartido.

6.3 Hoja de ruta para escalabilidad futura

Ante un eventual crecimiento masivo en la base de usuarios y volumen de envíos, la arquitectura actual encontraría cuellos de botella en la frecuencia del sondeo por *polling* y la carga sobre la persistencia. Para evolucionar el sistema, las mejoras prioritarias contempladas incluyen:

1. *Transición a eventos en vivo:* Sustituir el esquema de consultas recurrentes por una arquitectura de WebSockets para lograr actualizaciones de posición con menor latencia y consumo de red.
2. *Búfer de geolocalización en memoria:* Integrar una base de datos clave-valor (como Redis/Upstash) que actúe como ingesta temporal de coordenadas, consolidando los trayectos en PostgreSQL solo al finalizar el servicio.
3. *Desacoplamiento de módulos críticos:* Migrar progresivamente el motor cartográfico y los procesos analíticos hacia microservicios independientes, permitiendo escalar horizontalmente las áreas de mayor demanda sin afectar las funciones de soporte tecnico.
4. *Cancelaciones y reembolsos:* Incorporar el ciclo de cancelación de envíos y la devolución de pagos, que en esta versión no están contemplados (por ejemplo, cuando ningún transportista acepta un envío publicado o cuando el remitente desiste antes de la entrega). Se prevé agregar los estados “Cancelado” para el envío y “Reembolsado” para la transacción, un plazo máximo de espera de aceptación, la cancelación por parte del remitente o del transportista bajo reglas de negocio a definir, y la devolución del dinero retenido mediante la funcionalidad de reembolsos de Mercado Pago, registrando el resultado en la tabla Transacción.

Anexo A. Registro de decisiones arquitectónicas (ADR)

Este anexo resume, en formato ADR (contexto, decisión, alternativas y consecuencias), las decisiones que ya están justificadas en las secciones 2 a 6 de este documento. No incorpora decisiones nuevas.

ADR-01. Monolito full-stack organizado en capas

Contexto. Alcance acotado, equipo pequeño y despliegue serverless en Vercel. Se busca tipado compartido entre cliente y servidor.

Decisión. Un único repositorio y despliegue (Next.js) con frontend y API REST. Separación lógica en capas: middleware de seguridad, controladores, servicios y repositorios.

Alternativas. Microservicios separados (descartados por la sobrecarga operativa y de red para este alcance). Aplicaciones nativas por plataforma (descartadas en favor de una web responsive con un único despliegue).

Consecuencias. (+) Desarrollo ágil, despliegue simple y tipado compartido. (−) El backend no se puede escalar de forma independiente del frontend: si un módulo demanda recursos, se escala la aplicación completa.

Ver: Secciones 2.2.2, 2.3.1, 3 y 6.2.

ADR-02. PostgreSQL como única fuente de verdad, sin capa de caché

Contexto. Envío, Transacción y Transportista están muy interrelacionados y el negocio exige consistencia transaccional estricta (por ejemplo, que dos transportistas no acepten el mismo viaje).

Decisión. Base de datos relacional PostgreSQL accedida con Prisma. Las coordenadas GPS se escriben directamente en PostgreSQL, sin Redis/Upstash.

Alternativas. Base NoSQL (descartada: no ofrece de forma nativa las garantías ACID ni restricciones duras para la máquina de estados). Caché en memoria para el GPS (evaluada y descartada para esta etapa).

Consecuencias. (+) Una única fuente de verdad y menor complejidad operativa. (−) PostgreSQL absorbe todo el volumen de lecturas y escrituras; podría requerir escalar el hardware antes de lo previsto si crece la concurrencia.

Ver: Secciones 2.2.2, 3, 6.1 y 6.2.

ADR-03. Polling en lugar de WebSockets para el seguimiento GPS

Contexto. El seguimiento en vivo (RF03, CU-03) debe funcionar en un entorno serverless que no mantiene conexiones abiertas y con conectividad móvil inestable (RNF06).

Decisión. El cliente consulta la ubicación cada cierta cantidad de segundos (ej. 10 s) mediante REST. Ante un fallo de red, reintenta en el ciclo siguiente.

Alternativas. WebSockets (descartados: no son viables técnica ni económicamente en Vercel y se cortan fácilmente con micro-cortes 3G/4G).

Consecuencias. (+) Estabilidad y reconexión implícita. (−) Mayor cantidad de requests y más latencia que una arquitectura de eventos; es el principal cuello de botella a futuro.

Ver: Secciones 2.2.2, 4.1, 4.2 (RNF06) y 6.3.

ADR-04. Autenticación delegada en Clerk

Contexto. La gestión de credenciales, contraseñas y sesiones no forma parte del core del negocio y es una fuente de riesgo de seguridad.

Decisión. Clerk actúa como proveedor externo de identidad. El backend verifica los tokens JWT y no almacena contraseñas ni sesiones. El rol se obtiene de la metadata pública del usuario y, si está vacía, se asigna Remitente.

Alternativas. Desarrollar y mantener una autenticación propia (descartada por la complejidad y el riesgo de filtraciones).

Consecuencias. (+) Seguridad delegada en especialistas. (−) Dependencia de un tercero. Por las restricciones de la capa gratuita, el middleware debe consultar la API de Clerk para obtener el rol, lo que agrega una llamada externa al flujo de autorización.

Ver: Secciones 2.1.2, 4.2 (RNF05) y 5.1.

ADR-05. Procesamiento de pagos tercerizado en Mercado Pago

Contexto. El dinero atraviesa una máquina de estados (Retenido, Liberado, Liquidado) y procesarlo implicaría responsabilidades legales y de seguridad.

Decisión. El sistema no almacena dinero real ni funciona como billetera: orquesta los cobros y guarda referencias a transacciones externas. El cobro se confirma por webhook y las liquidaciones usan una idempotencyKey y reintentos con espera exponencial.

Alternativas. Procesar los pagos de forma propia o con una billetera virtual (descartado por el riesgo legal y los estándares de seguridad requeridos).

Consecuencias. (+) Menor riesgo legal y carga transaccional. (−) Dependencia de la disponibilidad de la API externa, que obliga a contemplar mecanismos de resiliencia.

Ver: Secciones 2.1.2, 5.2 y 5.3.

ADR-06. Procesos batch diferidos con Vercel Cron Jobs

Contexto. El cálculo de Zonas Calientes, las métricas globales y las liquidaciones consumen muchos recursos y no requieren respuesta inmediata (RNF02).

Decisión. Vercel Cron Jobs ejecuta los procesos en horarios de baja concurrencia (ej. 03:00), llamando a endpoints protegidos con un secreto. Un lock distribuido en PostgreSQL evita ejecuciones simultáneas.

Alternativas. Mantener un servidor encendido 24/7 de forma exclusiva para estas tareas (descartado por costo y operación).

Consecuencias. (+) Integración nativa con Next.js y sin impacto en la latencia diurna. (−) Los datos analíticos se actualizan solo una vez por día y un fallo del batch debe informarse al usuario.

Ver: Secciones 2.2.1, 4.2 (RNF02), 5.3 y casos de uso CU-08 y CU-10.

ADR-07. Soporte técnico limitado al Remitente en esta versión

Contexto. Permitir disputas bilaterales (por ejemplo, un conductor que reporta que el remitente no está en el domicilio) aumentaría la complejidad de los estados del envío.

Decisión. Solo el Remitente puede abrir tickets de soporte (RF05). Los Transportistas no inician reclamos en esta versión.

Alternativas. Habilitar también a los Transportistas para abrir tickets (postergado).

Consecuencias. (+) Alcance manejable para lanzar la primera versión. (−) Los Transportistas no cuentan con un canal propio de reclamo.

Ver: Sección 2.1.2 (Decisión 3).